Transformer Surrogate for Red-Black Gauss-Seidel on the 2D Poisson Equation

Oct 13, 2026 · @Sakthi

A decoder-only Transformer trained on GPU solver histories predicts the converged Poisson solution about 2 to 3 times better than freezing the checkpoint state, but a one-line Aitken extrapolation is about 150 times more accurate still.

Abstract

A Red-Black Gauss-Seidel solver (Numba CUDA, Kaggle Tesla T4) generates convergence histories of the 2D Poisson equation on a 50x50 grid. A causal Transformer, with one token per saved grid state, is trained on 80 of 100 such histories. Handed the history up to a checkpoint (relative residual 1e-3, about 48% of the iterations), it predicts the remainder of the solve. Predictions are compared with the HPC result, the analytical solution, and two non-learned baselines: freezing the checkpoint state and Aitken extrapolation. All numbers come from one executed run (seed 42).

Key findings

- **GPU vs CPU.** The GPU solver is 27x faster than pure-Python Gauss-Seidel but 5.4x slower than a compiled single-thread CPU loop at 50x50. It overtakes the CPU between N=100 and N=200 and reaches 17x at N=1600.
- **Jump to the converged state.** The Transformer beats freezing on 10 of 10 held-out problems (geometric-mean error ratio 0.42) and is 23x better on the unseen benchmark problem.
- **Aitken is stronger.** Aitken extrapolation is about 150x more accurate than the Transformer on the held-out set, and wins on every one of the 10 problems.
- **Rollout fails.** The step-by-step autoregressive rollout is worse than freezing on 8 of 10 problems.
- **Hybrid timing.** On the benchmark problem, warm-starting the solver from the AI jump saves 52% of the remaining iterations (hybrid 0.33 s vs 0.43 s for full CUDA, 1.29x). Averaged over the held-out problems it saves nothing (874 vs 999 iterations).

1. Problem and solvers

The task is to solve the Poisson equation on the unit square with zero boundaries, and to compare three solvers that all stop at the same relative residual of 1e-6.

Equation. $\nabla^2 u = f$ on $[0,1]^2$, 5-point stencil, N = 50 interior points per side (h = 1/51). Benchmark: $f = -2\pi^2 \sin(\pi x) \sin(\pi y)$, exact solution $u = \sin(\pi x) \sin(\pi y)$. The discrete solution differs from the exact one by about $\pi^2 h^2/12 \approx 3.2\text{e-}4$, which is the accuracy floor for any solver measured against the analytical solution.

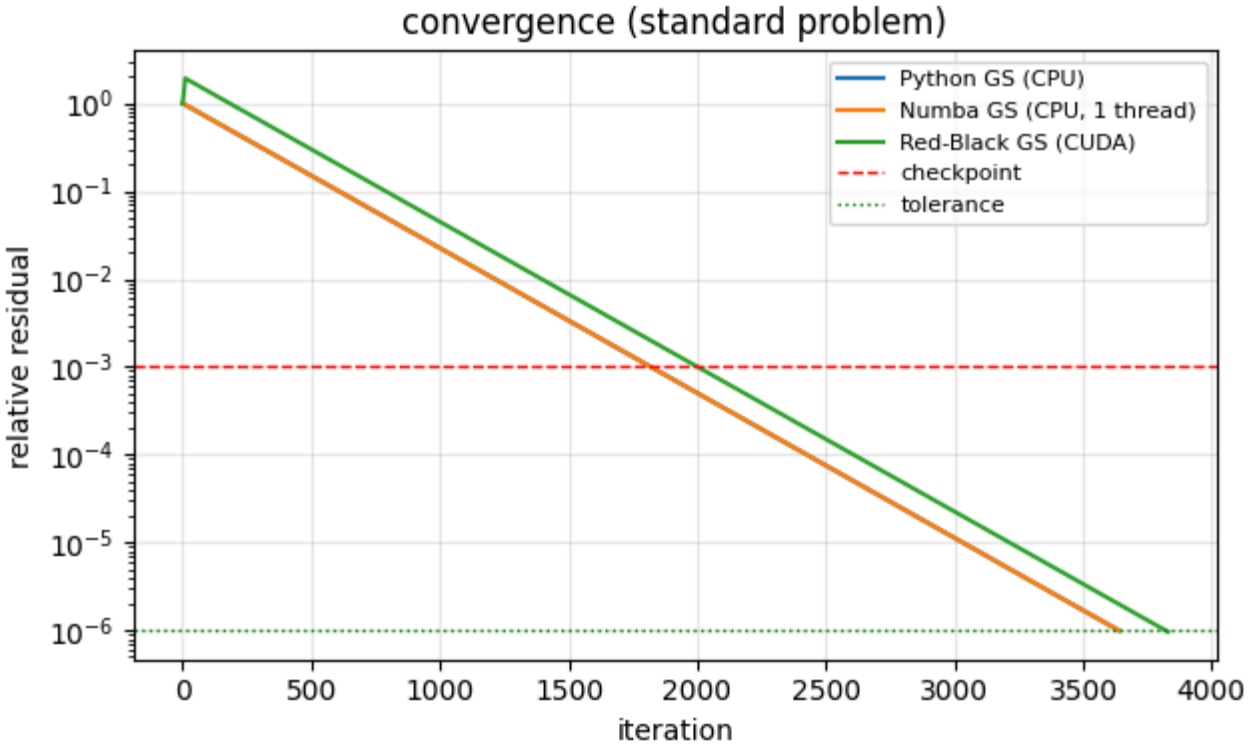
Stopping rule. Every solver evaluates the relative residual $\max|Au - f| / \max|f|$ every 10 iterations and stops at 1e-6.

Solvers. (a) pure-Python lexicographic Gauss-Seidel, (b) the same algorithm compiled with Numba on one CPU thread, (c) Red-Black Gauss-Seidel in Numba CUDA with one thread per cell and separate red and black kernels. The CUDA kernels were validated against a NumPy red-black reference (maximum difference 1.7e-16 after 40 iterations).

1.1 Solver comparison (benchmark problem, 50x50)

Solver	Iterations	Time	Max error vs analytical
Python GS (CPU)	3640	11.70 s	3.150e-4
Numba GS (CPU, 1 thread)	3640	0.080 s	3.150e-4
Red-Black GS (CUDA, T4)	3830	0.433 s	3.155e-4

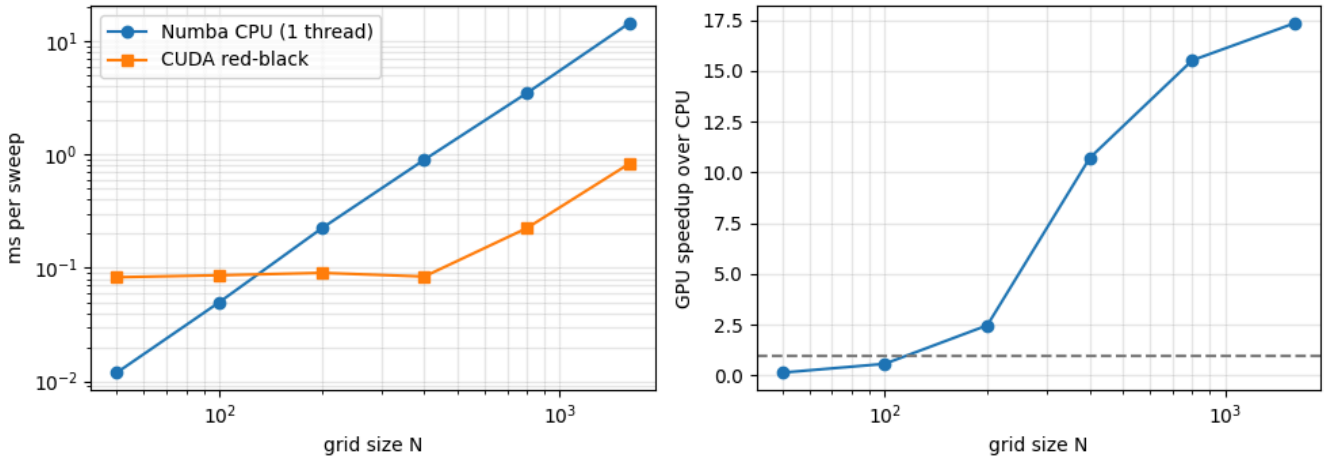
Red-black ordering needs about 5% more iterations than lexicographic ordering. The GPU is 27x faster than the pure-Python loop but 5.4x slower than the compiled CPU loop, because 2,500 unknowns do not keep a GPU busy.



1.2 Grid-size scaling (time per sweep)

N	CPU ms/sweep	GPU ms/sweep	GPU speed-up
50	0.0119	0.0829	0.14x
100	0.0499	0.0862	0.58x
200	0.2239	0.0906	2.47x
400	0.9008	0.0840	10.72x
800	3.4861	0.2246	15.52x
1600	14.4440	0.8323	17.35x

Up to N = 400 the GPU time per sweep is flat at about 0.085 ms, so it is limited by kernel-launch latency and not by arithmetic. The crossover with the CPU is between N = 100 and N = 200.



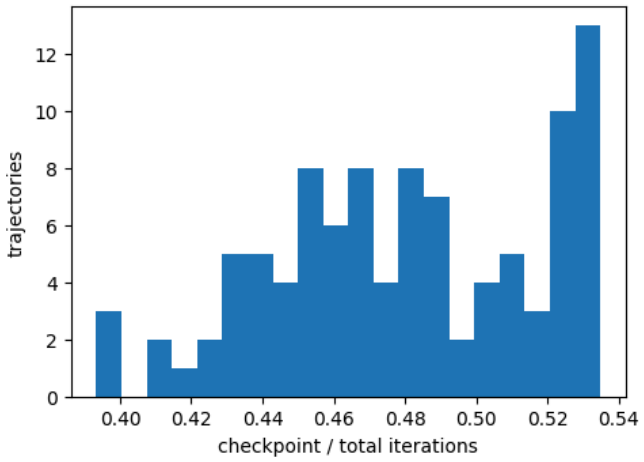
2. Dataset

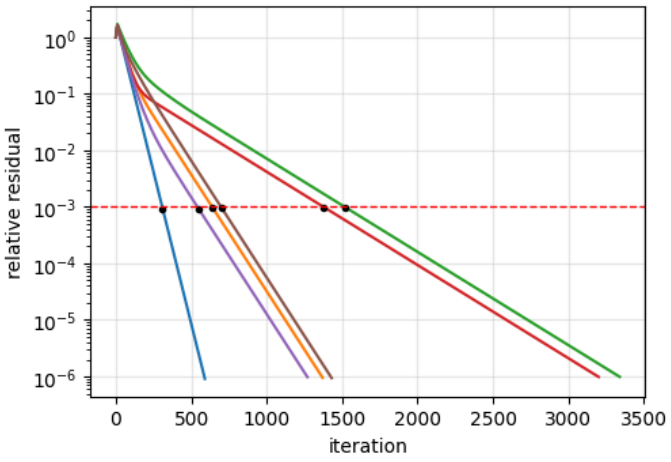
The training set is 100 random Poisson problems, each solved on the GPU with the full grid saved every 10 iterations; all 100 were accepted and generation took 27 s.

Each problem is a sum of sine modes with indices in {1, 2, 3} and random signs and amplitudes. Sixty percent include the slowest-converging (1,1) mode plus one to three others; the rest use one to three other modes. A pure (1,1) problem is never generated, so the benchmark is unseen. The solver starts from zero and saves the initial state plus a snapshot every 10 iterations until the residual reaches 1e-6.

- **Length:** 44 to 377 snapshots per trajectory, mean 220.
- **Checkpoint:** the first snapshot with relative residual ⩽ 1e-3, at 48% of the iterations on average (range 39% to 53%).

- **Split:** by trajectory, 80 train / 10 validation / 10 test, giving 16,392 training and 2,709 validation windows of 16 consecutive snapshots.
- **Normalisation:** each trajectory is divided by the maximum absolute value of its checkpoint state, which is known at hand-off time.





3. Model and training

The model is a 4.67M-parameter decoder-only Transformer that adds predicted changes to the current grid state; it stopped training at epoch 36 after 150 s, with the best epoch at 16.

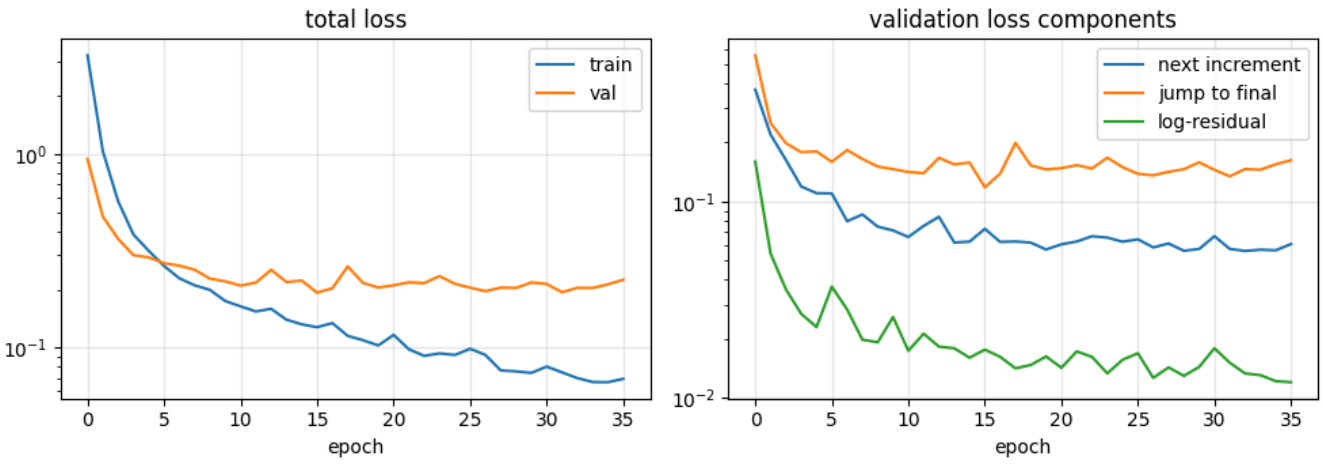
Tokens. One token per saved state: the normalised state and its increment since the previous snapshot, both flattened (5,000 inputs). The increment is compressed with asinh(x / 1e-4) so tiny late-stage changes remain visible.

Architecture. Linear projection to 256 dimensions plus LayerNorm, sinusoidal encoding of log(1 + iteration), four pre-LN decoder blocks (4 heads, feed-forward width 512, GELU, dropout 0.1), a causal mask and a final LayerNorm.

Heads (per position). (1) next increment, (2) remaining distance to the converged state (the "jump"), both in asinh space and inverted with sinh before being added to the current state, and (3) log-residual as an auxiliary output.

Training. AdamW (learning rate 5e-4, weight decay 1e-2, 3-epoch warm-up, cosine decay), batch size 64, gradient clipping at 1.0. Loss = next + jump + 0.1 x residual, with positions before the checkpoint weighted 0.3. Early stopping with patience 20 on validation loss.

Outcome. The best validation loss was 0.193. By epoch 35 the training loss (0.067) was about three times below validation (0.213), so the model overfits after roughly epoch 15; validation loss is flat from about epoch 10.



4. Results

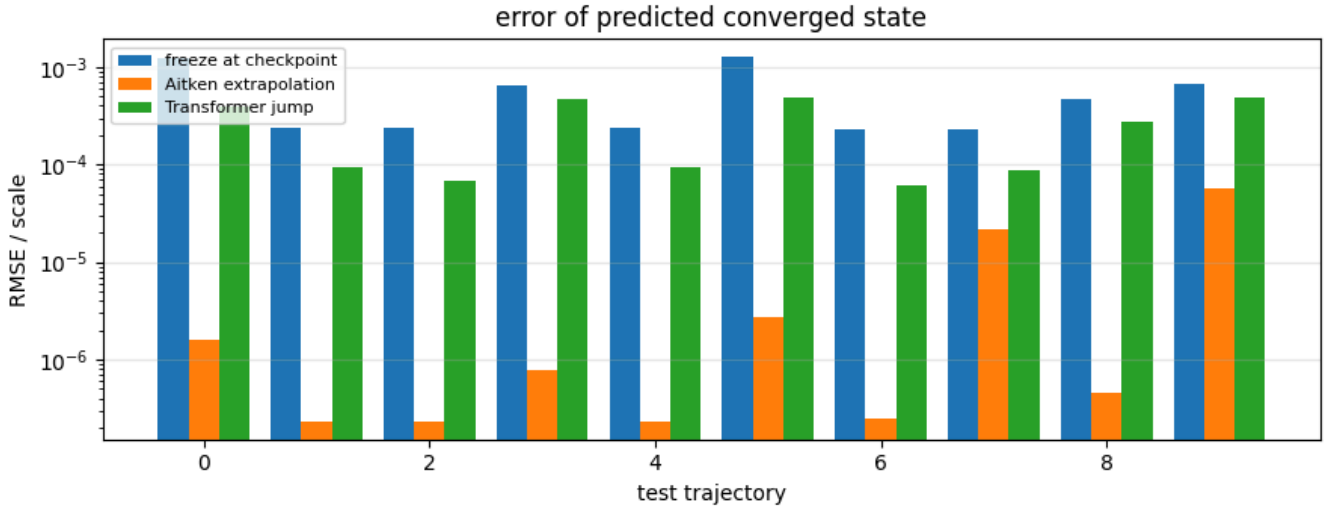
Errors are RMSE over the grid divided by the checkpoint scale, so 1e-3 means 0.1% of the solution amplitude. Two baselines need no training: *freeze* (take the checkpoint state as the answer) and *Aitken* ($\hat{u} = u_{ck} + \Delta\cdot\rho/(1-\rho)$, with Δ the last increment and one scalar ρ estimated from the last two increments).

4.1 Held-out problems: jump to the converged state

The Transformer beats freezing on all 10 problems but is about 150x less accurate than Aitken.

Method	Median error
Freeze at checkpoint	3.54e-4
Aitken extrapolation	6.17e-7
Transformer jump	1.87e-4

Transformer vs freeze: lower on 10/10, geometric-mean ratio 0.422. Transformer vs Aitken: higher on 10/10, geometric-mean ratio 152.9.



4.2 Autoregressive rollout

Averaged over the post-checkpoint snapshots, the rollout is worse than freezing on 8 of 10 problems (geometric-mean ratio 3.26). On the benchmark its error crosses above the freeze error at about iteration 2,850 and ends at 4.7e-4 versus 2.5e-4. Rolled-out states also have a large Laplacian residual, climbing to about 1e-1 even though their RMSE is small, because the Laplacian amplifies high-frequency error.

4.3 Does a better jump make the solver finish faster?

On average it does not: restarting from the Transformer jump needs more iterations than continuing from the checkpoint.

Start point	Iterations to reach 1e-6 (mean of 10 test problems)
Continue from the checkpoint	874
Restart from the Transformer jump	999
Restart from the Aitken estimate	85

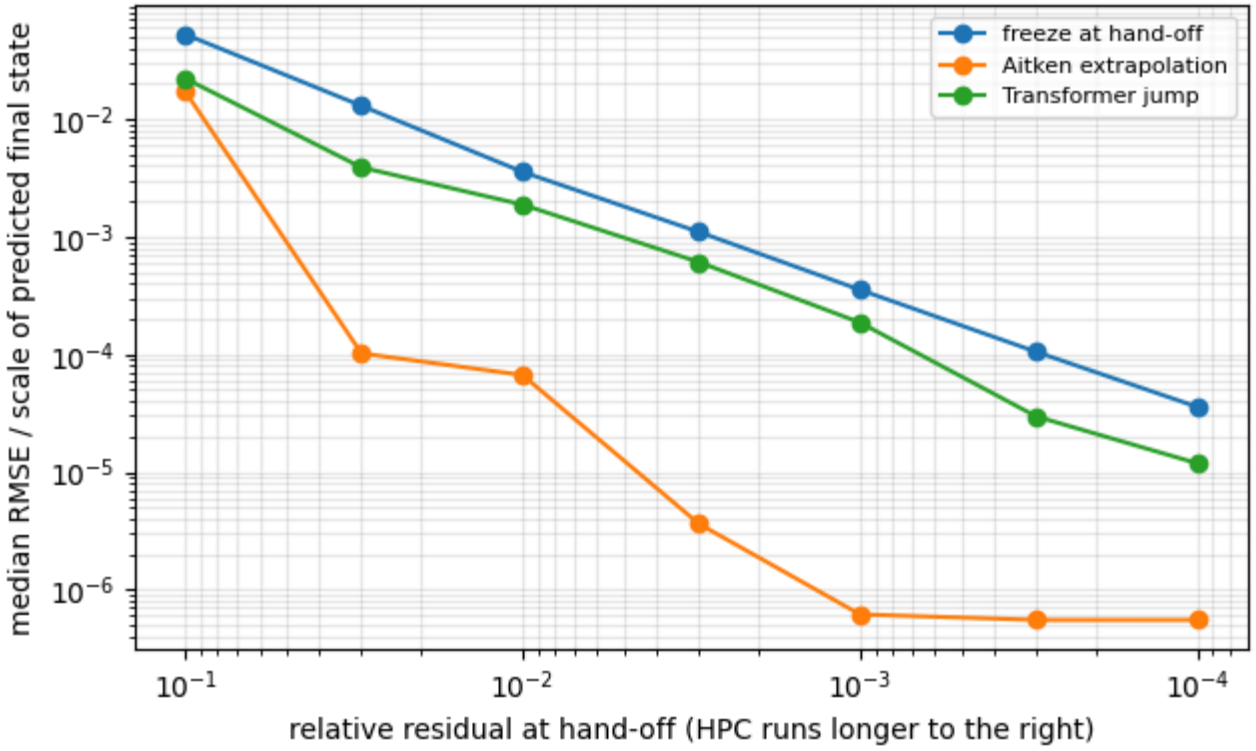
The jump has lower RMSE than freezing, but its error is not smooth and the solver's residual criterion penalises high-frequency components heavily. This explanation is a hypothesis consistent with the rollout residuals in 4.2; it was not tested separately.

4.4 Sensitivity to the hand-off point

The same trained model was evaluated when the solver is stopped at different relative residuals (median error over the test problems that reach that level).

Hand-off residual	n	Freeze	Aitken	Transformer	AI / freeze	AI / Aitken
1e-1	5	5.26e-2	1.73e-2	2.22e-2	0.42	1.3
3e-2	9	1.30e-2	1.02e-4	3.88e-3	0.30	38
1e-2	10	3.56e-3	6.68e-5	1.88e-3	0.53	28
3e-3	10	1.10e-3	3.63e-6	6.09e-4	0.55	168
1e-3	10	3.54e-4	6.17e-7	1.87e-4	0.53	303
3e-4	10	1.05e-4	5.54e-7	2.97e-5	0.28	54
1e-4	10	3.58e-5	5.53e-7	1.19e-5	0.33	21

The Transformer beats freezing at every hand-off (ratios 0.28 to 0.55). It comes closest to Aitken at the earliest hand-off (1e-1, only 5 problems), where several modes are still active, and still does not win.

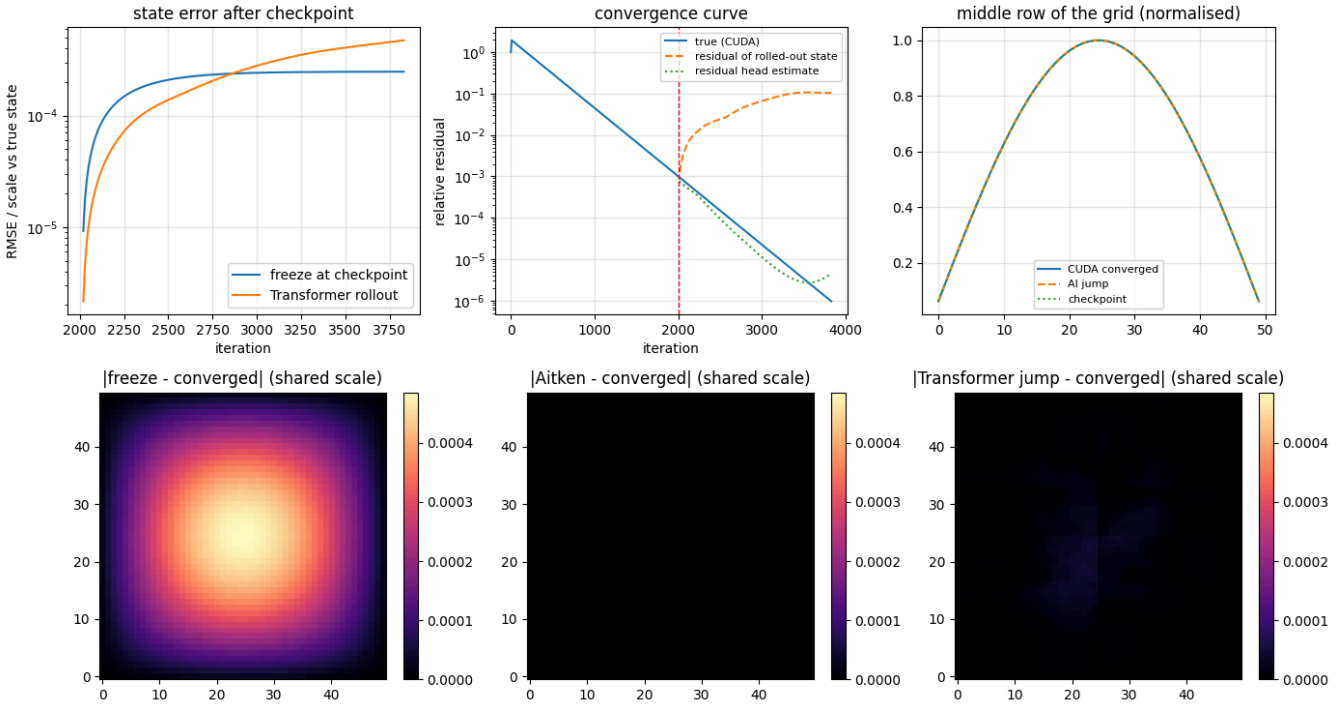


4.5 Benchmark problem: $\sin(\pi x)\sin(\pi y)$, unseen

The checkpoint is at iteration 2010 of 3830 (52% of the solve). The jump is 23x more accurate than freezing and about 44x less accurate than Aitken.

Quantity	Freeze	Aitken	Transformer jump
Error vs converged CUDA state	2.48e-4	2.47e-7	1.08e-5
Max error vs analytical	1.69e-4	3.16e-4	3.61e-4

The converged CUDA solution is 3.155e-4 from the analytical one, the discretisation floor. The frozen checkpoint looks closer to the analytical solution only because the unconverged iterate undershoots the discrete solution, which partly cancels the discretisation error. The rollout mean error is 2.52e-4 versus 2.13e-4 for freezing.



4.6 Hybrid timing (benchmark problem)

The hybrid run is 1.29x faster than full CUDA, saving 940 of the 1,820 post-checkpoint iterations. Timings are the median of 3 runs after JIT warm-up.

Step	Iterations	Time (s)
Full CUDA solve (zero to 1e-6)	3830	0.4286
CUDA to checkpoint (snapshots saved)	2010	0.2293
Finish from checkpoint without AI	1820	0.2080
Transformer jump (inference)	-	0.0038
CUDA warm-start from the AI jump to 1e-6	880	0.0987
Hybrid total	2890	**0.3318**

Final maximum error vs analytical is 3.165e-4 for the hybrid and 3.155e-4 for full CUDA. The full CUDA solve is itself about 5x slower than the compiled CPU solver at this size, so the gain is relative to the GPU solver, and the hybrid is still slower in absolute terms than the 0.080 s Numba CPU baseline.

5. Limitations

The first limitation can inflate the Transformer's results; the others limit how far they generalise.

1. **Duplicate problems and train/test overlap.** The solver starts from zero and is linear, so a problem and the same problem scaled by a constant give the same trajectory after amplitude normalisation. Only 8 distinct single-mode problems exist and about 13% of generated problems are single-mode, so repeats are likely. In the test results, problems 1 and 4 are identical in every column and problem 2 matches them on the baselines, which fits this explanation. The overlap probably favours the Transformer, and the effective test size is below 10. The fix is to split by mode content and use richer random forcing fields.
2. **Single run.** One seed, one split, one training run, and no confidence intervals. Hand-off levels with n = 5 or 9 are especially noisy.
3. **Narrow problem class.** Sums of up to four low-frequency sine modes on one grid size. A scalar-rate extrapolation is nearly exact for this class, which makes Aitken an unusually strong baseline.
4. **Timing scope.** The hybrid timing covers one problem and the GPU solver only. At this grid size launch latency dominates, so the speed-up at larger grids, where it matters, is not measured.
5. **Residual head.** It is trained on and evaluated against true states, while rolled-out states drift away from the training distribution.

6. Conclusions and next steps

The experiment supports four conclusions.

- A causal Transformer trained on solver histories predicts the converged state much better than freezing the checkpoint: typically 2x to 3x lower error, and 23x on the unseen benchmark.
- On this problem class it does not beat Aitken extrapolation, and its autoregressive rollout is not usable.
- Warm-starting the solver from the AI jump helps on the benchmark (52% fewer iterations) but not on average over the held-out problems.
- The GPU only pays off beyond roughly N = 100 to 200 against a compiled CPU loop.

Next steps, in order of expected value:

1. Remove the duplicate-problem overlap (split by mode content, use richer random forcing fields) and re-run with several seeds.
2. Use Aitken as the base and let the model learn the correction on top of it, targeting the multi-rate error that extrapolation misses, which is most visible at early hand-offs.
3. Evaluate on problems where classical extrapolation is weak: more modes, non-separable forcing, variable coefficients, or nonlinear PDEs.
4. Measure the hybrid at larger grids (N = 400 to 1600), where each iteration is expensive and a saved iteration is worth more.
5. Penalise the residual of the predicted state, or add a smoothness loss, so a lower RMSE also gives a lower residual, and train with a multi-step rollout loss to reduce drift.

7. Reproducibility

Everything ran on a Kaggle Tesla T4 (15.6 GB) with PyTorch 2.10.0+cu128 and Numba 0.60.0, seed 42, in one notebook run top to bottom. The HPC solvers are deterministic; the trained model may vary slightly between runs because of GPU nondeterminism. Runtime on the T4: solver baselines and scaling study about 1 minute, dataset generation 27 s, training 150 s, evaluation and timing a few seconds. The notebook writes `results.json`, the best model weights and the benchmark figure to the output folder.